

DELIVERABLE 1.1

Framework Selection Report

Pangasinan Heritage Digital Showcase

Current implementation decision and rationale

Updated for the current Next.js application

Selected stack: React with Next.js 14

The delivered application uses **Next.js 14.2.35 with the App Router**, React 18, TypeScript 5, and Tailwind CSS 3.4.1. This choice supports the project's image-forward tourism experience while keeping a clear file-based structure for routes, reusable components, and future destination content.

Decision	Implemented evidence
Framework	Next.js 14.2.35 with App Router routes under src/app.
UI runtime	React 18 function components; client boundaries only where state, DOM events, observers, or portals are needed.
Styling	Tailwind CSS 3.4.1 with a custom sea, sand, coral, moss, and ink theme.
Media	next/image with explicit sizes and priority heroes; next/font/google for three local font variables.
Icons	lucide-react for the expanded editorial set, plus a small local SVG Icon atom.
Validation	npm run build completed compilation and type validation for the current source.

The recommendation is confirmed by the implemented code, not just a theoretical comparison: Next.js provides the production structure now in use.

React + Next.js 14 versus Vue + Nuxt 3

Criterion	React + Next.js 14	Vue + Nuxt 3	Assessment for this project
Route and layout model	App Router supplies route folders, metadata, and layouts close to the UI structure.	File-based routing and layouts are equally capable.	Next is already implemented across four public routes.
Static and server-first delivery	Server Components support static presentation by default; dynamic behavior is isolated with use client.	Nuxt has strong SSG and hybrid-rendering options.	The current atom/page split maps cleanly to Next client boundaries.
Image and font tooling	next/image and next/font are integrated into delivered routes and root layout.	Nuxt Image and font modules can meet the need with setup.	Next has zero additional framework configuration in the current build.
Component architecture	TypeScript React components directly reflect Atomic Design directories.	Vue SFCs and Composition API also work well for Atomic Design.	React convention is already established in source and documentation.
Interactive UI	Client components handle search, filters, modal portal, mobile navigation, and scroll reveal.	Vue reactive state is concise and capable.	No migration or duplicated patterns are necessary with Next.
Ecosystem and handoff	Large React ecosystem; common TypeScript and accessibility tooling.	Mature ecosystem with a smaller React-compatible library pool.	React/Next reduces handoff friction for this delivered codebase.

Implementation evidence

Requirement	How the implementation addresses it
Mobile-first presentation	Tailwind base styles begin with narrow layouts; sm, md, lg, and xl progressively add columns, horizontal actions, navigation, and header search.
Fast media handling	Hero images declare priority and all image components specify responsive sizes. next/image can optimize delivered image variants in a configured production deployment.
Maintainable content	Four destinations are centralized in src/lib/heritage-sites.ts. Search, gallery, cards, and modal detail reuse the same typed records.
Accessible interaction	Visible focus styles, labels for search and icon-only controls, route state via aria-current, modal semantics, Escape close, and reduced-motion handling are implemented.
Clear component hierarchy	Atoms, molecules, and organisms live in named directories; route components compose them with page-local editorial blocks.
Production check	The current project runs next build successfully, compiling the optimized application and validating types.

Client boundary choices

Next's Server and Client Component model is used pragmatically. Presentation components such as the global layout and basic atoms can remain server-rendered. State-bearing and browser-dependent organisms use "use client": HeaderNavigation, SearchForm, HeritageGallery, HeritageGrid, TipCard, and DestinationDetailModal. This keeps interactive concerns explicit and avoids pushing every route into a single all-client architecture.

Trade-offs acknowledged

Nuxt 3 would be a technically credible alternative, especially for a team standardised on Vue. The principal cost of changing now would be implementation churn: routes, component contracts, image behavior, interaction code, and the documentation would all need rework without solving a demonstrated project need. The existing Next.js solution is coherent, compiles successfully, and meets the requested design direction.

4. FINAL DECISION

Keep React + Next.js 14 for this project

React with Next.js 14 remains the selected framework for the Pangasinan Heritage Digital Showcase. It is the framework already delivering the revised visual design and features: immersive route heroes, responsive editorial grids, optimized image components, route-aware navigation, destination search, filters, and a modal detail experience.

Recommended next steps

Priority	Action
1	Run the deployed production build behind the intended image host and check page performance on representative mobile networks.
2	Test the search results, gallery filters, navigation panel, and modal by keyboard and with a screen reader on mobile and desktop.
3	Add content governance for image licensing, destination descriptions, and tourism-office contact details before public launch.
4	If more destinations are planned, extend the shared data schema and consider a CMS only when editorial update frequency justifies it.

Framework baseline

Next.js 14.2.35 | React 18 | TypeScript 5 | Tailwind CSS 3.4.1 | lucide-react 1.35.0